

Sound-Based Machine Health Monitor

Software Requirements Specification (SRS)
A Comprehensive Technical Specification for University Project
Implementation

Document Version: 1.0

Document Status: Draft

Date of Preparation: September 24, 2026

Prepared by: [Student Name]

University / Institution: [University Name]

Department: [Department Name]

Course: [Course Name]

Table of Contents

1. Introduction.....	3
2. Overall Description.....	4
3. System Features and Functional Requirements.....	5
4. Non-Functional Requirements.....	9
5. External Interface Requirements.....	10
6. System Architecture and Design Overview.....	11
7. Data Requirements.....	13
8. AI/ML Requirements.....	14
9. Error Handling and Exception Requirements.....	15
10. Testing and Acceptance Requirements.....	16
11. Requirement Traceability Matrix.....	17
12. Project Constraints and Future Scope.....	18
13. Appendices.....	19

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document details the functional and non-functional specifications for the Sound-Based Machine Health Monitor. It serves as the primary technical blueprint for developers, AI/ML engineers, testers, and project evaluators. By translating the operational vision from the Product Requirements Document (PRD) into precise specifications, this document governs implementation boundaries, testing protocols, and acceptance benchmarks.

1.2 Document Scope

The system covered by this specification is a prototype application combining audio signal processing and machine learning to classify machine operating sounds into categories such as normal, abnormal, or uncertain. The initial release is scoped to handle single-clip audio uploads and browser-based microphone recordings, execute validation and feature extraction, perform model inference, display results on a responsive dashboard, and maintain a local SQLite-backed analysis history. Real-time streaming, automated IoT hardware integration, and multi-user cloud authentication are explicitly excluded from this initial version.

1.3 Project Overview

The Sound-Based Machine Health Monitor addresses the economic and logistical burdens of unplanned machine downtime. Manual inspections and rigid time-based maintenance models are inefficient or resource-intensive. By leveraging the principle that machines emit distinct acoustic signatures whose deviations indicate developing faults, this software offers a low-cost, accessible tool for acoustic condition monitoring using standard computing resources.

1.4 Intended Audience

- **Developers:** Responsible for building the backend REST API, database connectors, and frontend components.
- **AI/ML Engineers:** Responsible for audio preprocessing pipelines, feature engineering, model training, and evaluation.
- **Testers:** Responsible for executing functional, integration, negative, and performance test suites.
- **Project Supervisors & University Evaluators:** Responsible for grading, academic compliance, and architectural review.
- **End Users:** Factory technicians or workshop owners interacting with the application interface.

1.5 Definitions, Acronyms, and Abbreviations

- **SRS:** Software Requirements Specification
- **PRD:** Product Requirements Document

- **AI / ML:** Artificial Intelligence / Machine Learning
- **API:** Application Programming Interface
- **MFCC:** Mel-Frequency Cepstral Coefficients
- **CNN:** Convolutional Neural Network
- **Audio Preprocessing:** The workflow of resampling, silence trimming, and volume normalization.
- **Feature Extraction:** Converting raw audio waveforms into structured numeric arrays or spectrogram images.
- **Classification:** Categorizing an extracted feature representation into a target class label.
- **Anomaly Detection:** Identifying data points that deviate significantly from a baseline of normal behavior.

1.6 References

- Uploaded Product Requirements Document (PRD.md)
- Publicly available industrial sound datasets (e.g., MIMII dataset references)

1.7 Document Conventions

Functional requirements are designated as FR-001, FR-002, etc. Non-functional requirements are designated as NFR-001, NFR-002, etc. Requirements are marked as Mandatory, Optional, or Future Enhancement based on their criticality to the prototype baseline. 'The system' refers exclusively to the Sound-Based Machine Health Monitor application.

2. Overall Description

2.1 Product Perspective

The application operates as a standalone web-based prototype comprising a frontend user interface, a backend API server, an internal audio processing and machine learning inference engine, and a lightweight local database. It stands as an independent demonstration tool for predictive maintenance concepts without dependency on expensive industrial IoT infrastructures.

2.2 Product Functions

- Audio ingestion via file upload or in-browser microphone capture.
- Automated audio validation and preprocessing (resampling, silence trimming, volume normalization).
- Acoustic feature extraction (MFCCs, spectral properties, or spectrogram generation).
- Machine learning model inference yielding a classification label and numerical confidence score.
- Visual dashboard representation featuring health status indicators and waveform/spectrogram plots.
- Historical logging and retrieval of past analysis records using SQLite.

2.3 User Classes and Characteristics

- Factory Maintenance Technician ('Raj'): Possesses practical machinery experience but limited time; requires quick, dependable health status checks from short audio clips.
- Small Manufacturing Business Owner ('Simran'): Manages a small workshop; seeks an affordable, straightforward tool to prevent catastrophic breakdowns without specialized acoustic engineering software.
- University Student / Reviewer: Technically proficient; evaluates the codebase, methodology, accuracy metrics, and academic rigor.

2.4 Operating Environment

- Operating Systems: Windows 10/11, macOS, or Linux (standard student laptop environments).
- Web Browsers: Modern versions of Google Chrome, Mozilla Firefox, or Microsoft Edge supporting HTML5 audio recording APIs.
- Runtime & Environment: Python 3.10+ execution environment for backend services.
- Database: Local SQLite database engine.
- Hardware: Standard CPU-based laptop configuration with standard microphone hardware inputs.

2.5 Design and Implementation Constraints

- Dataset Availability: Constrained by open-source public audio datasets and their inherent class distributions.
- Acoustic Quality: Live audio capture is subject to ambient factory noise and microphone variance.
- Computational Resources: Execution is restricted to CPU-based inference without dedicated hardware accelerators.
- Timeline: Bounded by academic semester schedules and student resource limits.

2.6 User Documentation

The system will feature an intuitive, self-explanatory UI layout complemented by an overview section on the homepage and a comprehensive project documentation repository (docs/) detailing setup, API usage, and evaluation metrics.

2.7 Assumptions and Dependencies

- Assumption 1: Users will provide audio clips containing recognizable operational sounds from supported machine types.
- Assumption 2: Modern browsers will grant microphone permissions when requested for live recording features.
- Dependency: Relies on standard Python data science and audio libraries (e.g., librosa, scikit-learn, FastAPI).

3. System Features and Functional Requirements

3.1 Audio Input Management

- FR-001 (Audio file upload): User can upload an audio file through the dashboard interface. [Mandatory]
- FR-002 (Microphone recording): User can record audio directly inside the browser using microphone controls. [Optional]
- FR-003 (Format validation): System rejects unsupported audio formats with clear messaging. [Mandatory]
- FR-004 (Duration and size validation): System enforces maximum duration and file size constraints. [Mandatory]

3.2 Audio Preprocessing

- FR-005 (Audio preprocessing execution): System resamples audio, trims trailing/leading silence, and normalizes volume levels prior to feature extraction. [Mandatory]

3.3 Feature Extraction

- FR-006 (Feature extraction): System computes MFCCs, spectral features, and/or Mel-spectrogram representations consistently using parameters identical to model training. [Mandatory]

3.4 Machine Learning Classification

- FR-007 (ML inference execution): System passes processed audio features to the trained classification model to generate probability distributions. [Mandatory]
- FR-008 (Normal/abnormal classification): System evaluates confidence scores against thresholds to output a definitive class label (normal, abnormal, or uncertain). [Mandatory]
- FR-009 (Confidence display): System calculates and outputs a numeric confidence/uncertainty value alongside the prediction. [Mandatory]

3.5 Machine Health Result Display

- FR-010 (Health status visualization): Dashboard shows a clear visual status indicator corresponding to classification outcomes (color-coded badges). [Mandatory]
- FR-011 (Waveform and spectrogram visualization): Dashboard displays a graphical plot of the analyzed audio waveform or spectrogram. [Optional]

3.6 Dashboard and History Management

- FR-012 (Analysis history tracking): Past analyses are persistently logged and listed with timestamps and results. [Optional]
- FR-014 (Error messaging): Provides user-friendly error notifications for all failure cases without exposing stack traces. [Mandatory]

- FR-015 (Responsive dashboard layout): User interface adapts fluidly to common screen sizes and viewport widths. [Optional]
- FR-016 (Sample audio demos): Preloads example audio clips for instant demonstration testing. [Optional]

4. Non-Functional Requirements

The non-functional requirements governing system quality attributes are summarized below:

ID	Category	Requirement Description & Target
NFR-001	Performance	End-to-end prediction response delivered in under 5 seconds on a standard student laptop.
NFR-002	Reliability	System must handle malformed or invalid inputs gracefully without unexpected backend crashes.
NFR-003	Usability	A first-time user must be able to complete an audio upload and review results within minutes without training manuals.
NFR-004	Maintainability	Source code must be cleanly modularized into separated components (API, preprocessing, inference, UI).
NFR-005	Scalability	Prototype architecture must maintain clean separation of concerns to avoid blocking future scaling.
NFR-006	Compatibility	Must execute successfully across Windows, macOS, and Linux environments running Python 3.10+.
NFR-007	Security	Uploaded files must undergo strict validation before processing to prevent arbitrary code execution vulnerabilities.
NFR-008	Privacy	System must not collect unnecessary personal user data; audio retention policies must be transparently observed.
NFR-009	Accessibility	User interface must maintain readable contrast ratios and proper element labeling.
NFR-010	Error Recovery	Pipeline execution errors must not corrupt stored history records or crash the backend server service.

5. External Interface Requirements

5.1 User Interface

Main Dashboard featuring an introductory panel, file drop zone, recording controls, progress spinner, results display card, and history drawer. Error messages utilize clear modal banners or alert boxes detailing specific failure reasons.

5.2 Hardware Interfaces

Standard computer microphone hardware for live in-browser audio recording, and standard audio output devices for listening to preloaded demo samples.

5.3 Software Interfaces

- Frontend-Backend Interface: RESTful HTTP JSON APIs communicating between the web browser and the backend server.
- Database Interface: SQLite connector library managing local file-based database queries and updates.
- ML Model Interface: Serialized model loader interface executed within the Python backend runtime.

5.4 Communication Interfaces

Standard HTTP/HTTPS protocol requests utilized for client-server communication across local development ports or deployment targets.

6. System Architecture and Design Overview

6.1 High-Level Architecture

The application follows a decoupled client-server pattern. The frontend web interface submits audio files or recordings via HTTP POST requests to the backend API. The backend processes the audio stream, extracts features, performs machine learning inference, logs execution details to SQLite, and returns a structured JSON payload.

6.2 Component Description

- Frontend UI: HTML5/CSS/JavaScript interface handling user interaction and visualization.
- Backend API (FastAPI): Exposes endpoints for analysis requests and history retrieval.
- Audio Preprocessing Module: Handles file validation, format conversion, resampling, silence removal, and normalization.
- ML Inference Module: Loads pre-trained classifier and executes prediction routines.
- Database Module: SQLite storage layer maintaining past analysis records.

6.3 System Workflow

[User / Audio Input] —> [Validation Layer] —> [Preprocessing Module] —> [Feature Extraction] —> [ML Model Inference] —> [Uncertainty Check] —> [Dashboard Result Display & History Storage]

6.4 Architecture Diagram

Client Browser (UI) —(HTTP POST)—> FastAPI Backend Server —> Preprocessing & Feature Extraction —> ML Inference Engine —> SQLite Database.

6.5 Data Flow Diagram

1. Client captures/selects audio and sends multipart form data to /api/analyze.
2. Backend validates properties and rejects malformed inputs immediately with error codes.
3. Valid audio streams pass through resampling and feature extraction routines.
4. Extracted features are fed into the serialized ML model, returning probabilities.
5. System applies threshold rules, writes a record to SQLite, and returns a JSON object.

7. Data Requirements

7.1 Input Data

Uncompressed or standard compressed audio container formats (WAV, MP3) sampled at standard rates (22.05 kHz mono) with clip durations between 1 to 10 seconds.

7.2 Dataset Requirements

Sourced from publicly available industrial machine sound datasets (e.g., MIMII). Contains labeled normal and abnormal operational recordings split using strict stratified training, validation, and test splits (70/15/15) to prevent data leakage.

7.3 Database Requirements

The SQLite database contains a single primary table named analyses:

Field Name	Data Type	Purpose	Constraints
analysis_id	INTEGER	Primary key identifier for the record	Primary Key, Auto-increment
timestamp	DATETIME	Records exact time execution took place	Not Null

machine_category	TEXT	Optional user-tagged machine category type	Nullable
audio_reference	TEXT	Stored filename or reference hash of processed clip	Not Null
predicted_class	TEXT	Final classification output (normal/abnormal/uncertain)	Not Null
confidence	FLOAT	Model confidence probability score	Not Null
processing_status	TEXT	Execution status flag (success / error)	Not Null
result_summary	TEXT	Human-readable summary text	Nullable

7.4 Data Retention

Raw uploaded audio files are treated as temporary assets and purged immediately after processing completes. Only structured textual metadata and analysis metrics persist inside the SQLite database.

7.5 Data Integrity

All incoming payloads are sanitized and validated at API entry points. Database transactions employ error catching to ensure logging failures do not interrupt client response delivery.

8. AI/ML Requirements

8.1 Machine Learning Objective

Supervised binary classification (normal vs. abnormal machine sound) supplemented by an uncertainty fallback threshold for ambiguous predictions.

8.2 Model Input and Output

Input representation consists of fixed-size feature vectors composed of aggregated MFCC statistical moments or standardized Mel-spectrogram tensor dimensions. Output classes map to normal, abnormal, or uncertain alongside a floating-point confidence score.

8.3 Audio Processing Pipeline

Raw waveforms are loaded, resampled, truncated/padded to fixed durations, stripped of silence, and normalized in amplitude before feature transformation.

8.4 Model Training

Models are trained offline using extracted feature sets derived from curated public datasets with hyperparameter tuning evaluated across cross-validation folds.

8.5 Model Evaluation

Evaluated on held-out test datasets using accuracy, precision, recall, F1-score (focusing heavily on minority abnormal classes to minimize false negatives), and confusion matrix analysis.

8.6 Model Inference

The backend loads the serialized model artifact into memory, executing fast CPU-based inference on incoming feature vectors to return real-time predictions.

8.7 Uncertainty and Limitations

Predictions failing to meet confidence probability thresholds are labeled as uncertain. Generalization is bounded by the diversity of the training dataset.

9. Error Handling and Exception Requirements

The system defines robust exception handling mechanisms across all pipeline stages:

Error ID	Error Condition	System Response	User Message	Recovery
ERR-001	Unsupported format	Reject before preprocessing	Unsupported file format. Upload WAV/MP3.	Prompt valid format
ERR-002	Empty/silent audio	Reject during validation	Audio appears silent or empty.	Request new recording
ERR-003	Corrupted audio	Catch decode exception	Unable to read audio file.	Prompt alternative file
ERR-004	Excessive duration	Reject/truncate per limits	Audio exceeds max duration.	Instruct user to trim

ERR-005	Mic permission denied	Catch permission rejection	Microphone access denied.	Display guidance
ERR-006	Missing model file	Halt service startup	Model artifact not found.	Log critical error
ERR-007	Inference failure	Catch runtime error	Analysis failed. Please try again.	Allow user to retry
ERR-008	Database failure	Bypass logging; return result	History could not be saved.	Return result to user

10. Testing and Acceptance Requirements

- Functional Testing: Verification of requirements FR-001 through FR-016 through direct UI and API testing.
- Integration Testing: End-to-end test suites validating data flow across frontend, backend, ML engine, and database.
- AI/ML Testing: Evaluation scripts running against held-out test datasets to confirm accuracy, precision, recall, and confusion matrices.
- Negative Testing: Resilience testing using malformed inputs, corrupted files, and permission denials.
- Performance Testing: Benchmarking response times to ensure end-to-end classification completes under the 5-second target.
- Acceptance Criteria: Valid audio input yields classification within target time; negative errors trigger correct messages without crashes; academic metrics met.

11. Requirement Traceability Matrix

The traceability matrix connects PRD features to SRS requirements, system modules, and test cases:

PRD Feature	SRS ID	System Module	Test Case ID
Audio file upload	FR-001	Frontend / API	TC-F-001
Microphone recording	FR-002	Frontend UI	TC-F-002
Format validation	FR-003	Validation Module	TC-N-001

Duration/size validation	FR-004	Validation Module	TC-N-002
Audio preprocessing	FR-005	Preprocessing Module	TC-I-001
Feature extraction	FR-006	Feature Extractor	TC-I-002
ML inference	FR-007	Inference Engine	TC-ML-001
Classification label	FR-008	Inference Engine	TC-ML-002
Confidence display	FR-009	Frontend / API	TC-F-003
Health visualization	FR-010	Frontend UI	TC-F-004
Waveform/spectrogram plot	FR-011	Frontend UI	TC-F-005
Analysis history	FR-012	Database Module	TC-I-003
Error messages	FR-014	API / UI	TC-N-003
Responsive dashboard	FR-015	Frontend UI	TC-F-006
Sample audio demos	FR-016	Frontend UI	TC-F-007

12. Project Constraints and Future Scope

12.1 Project Constraints

Limited to student-level hardware resources (CPU-only laptop execution), reliance on publicly accessible datasets without direct factory sensor integration, and sensitivity of live acoustic recordings to uncontrolled ambient background noise.

12.2 Known Limitations

The prototype cannot perform granular, machine-specific fault diagnosis beyond general normal/abnormal classification. Model performance is bounded by training dataset variety.

12.3 Future Enhancements

Continuous streaming and IoT sensor integration, multi-machine dashboard management, granular fault categorization, remaining-useful-life (RUL) estimation, and edge-device deployment.

13. Appendices

Appendix A: Requirement ID Summary

Functional Requirements: FR-001 through FR-016

Non-Functional Requirements: NFR-001 through NFR-010

Error Conditions: ERR-001 through ERR-008

Appendix B: Important Terminology

Acoustic Anomaly: Deviation from established normal operating sound patterns.

MFCC: Mel-Frequency Cepstral Coefficients representing spectral timbre.

Data Leakage: Improper inclusion of evaluation information within training subsets.

Appendix C: Assumptions

Users possess basic web navigation competency. Standard microphone permissions are accessible via supported modern web browsers.

Appendix D: Document Review Checklist

1. PRD analyzed and reflected accurately? [Yes]
2. Unique requirement IDs assigned? [Yes]
3. Test criteria mapped cleanly? [Yes]
4. Future scope separated from initial baseline? [Yes]